Advanced Machine Learning

Prof. Dr. Alexander Dück

alexander.dueck@dhbw.de

Aufbau der Vorlesung

Kurze Wiederholung

Maschinelles Lernen (ca. 60%)

Reinforcement Learning (ca. 40%)

Zwischendrin Zeit für Praxis

Vektoren, Matrizen, Gradienten

Vektoren und Matrizen

- Matrizen:
- $A = \begin{pmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{pmatrix} \in \mathbb{R}^{2x3}, \qquad M = \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \\ M_{31} & M_{32} \\ M_{41} & M_{42} \end{pmatrix} \in \mathbb{R}^{4x2}$
- Matrixelement A_{ij}; $\quad i$: Zeilenindex, $\quad j$: Spaltenindex
- Vektoren: $x = \begin{pmatrix} x_1 & x_2 & x_3 \end{pmatrix} \in \mathbb{R}^{1x3}, \qquad y = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} \in \mathbb{R}^{3x1}$

Matrix-Multiplikation:

- $$\begin{pmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{pmatrix} \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \\ M_{31} & M_{32} \end{pmatrix}$$
$$= \begin{pmatrix} A_{11}M_{11} + A_{12}M_{21} + A_{13}M_{31} & \dots \\ \dots & \dots \end{pmatrix}$$

- „Zeile auf Spalte“

Matrix-Multiplikation:

- $$\begin{pmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{pmatrix} \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \\ M_{31} & M_{32} \end{pmatrix}$$

$$= \begin{pmatrix} A_{11}M_{11} + A_{12}M_{21} + A_{13}M_{31} & \dots \\ A_{21}M_{11} + A_{22}M_{21} + A_{23}M_{31} & \dots \end{pmatrix}$$

Matrix-Multiplikation:

- $$\begin{pmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{pmatrix} \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \\ M_{31} & M_{32} \end{pmatrix}$$

$$= \begin{pmatrix} A_{11}M_{11} + A_{12}M_{21} + A_{13}M_{31} & A_{11}M_{12} + A_{12}M_{22} + A_{13}M_{32} \\ A_{21}M_{11} + A_{22}M_{21} + A_{23}M_{31} & \dots \end{pmatrix}$$

Matrix-Multiplikation:

- $$\begin{pmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{pmatrix} \begin{pmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \\ M_{31} & M_{32} \end{pmatrix}$$

$$= \begin{pmatrix} A_{11}M_{11} + A_{12}M_{21} + A_{13}M_{31} & A_{11}M_{12} + A_{12}M_{22} + A_{13}M_{32} \\ A_{21}M_{11} + A_{22}M_{21} + A_{23}M_{31} & A_{21}M_{12} + A_{22}M_{22} + A_{23}M_{32} \end{pmatrix}$$

Matrix-Produkt AM definiert für $A \in \mathbb{R}^{nxm}$ und $M \in \mathbb{R}^{mxk}$, d.h.
#Spalten(linke Matrix) = #Zeilen(re. Matrix) = m:

$$(AM)_{ij} = \sum_{l=1}^{m} A_{il} M_{lj}$$

Matrix-Multiplikation in Komponentenschreibweise

Spezialfall: Matrix x Vektor:

$$\begin{pmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} M_{11}x_1 + M_{12}x_2 + M_{13}x_3 \\ M_{21}x_1 + M_{22}x_2 + M_{23}x_3 \\ M_{31}x_1 + M_{32}x_2 + M_{33}x_3 \end{pmatrix}$$

Spezialfall der Matrixmultiplikation, mit Dimension der rechten Matrix = $\mathbb{R}^{3x1}$.

Ferner kann das Skalarprodukt zweier Vektoren $\theta = \begin{pmatrix} \theta_1 \\ \vdots \\ \theta_n \end{pmatrix}$ und $x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$ als Matrixmultiplitaktion geschrieben werden:

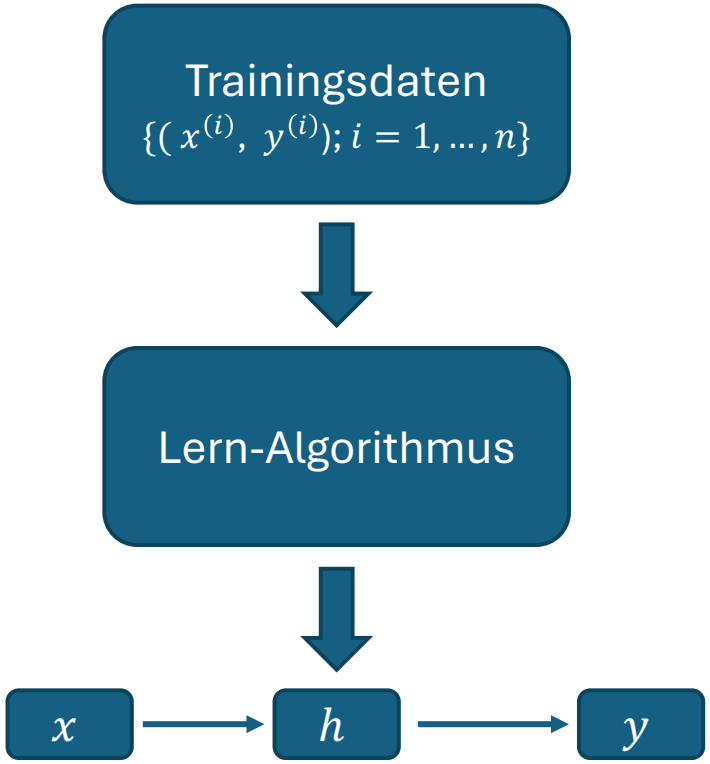
$$\theta^T x = \theta_1 x_1 + \cdots + \theta_n x_n$$

Partielle Ableitungen

- Bei Funktionen mehrerer Variablen $\theta_0, \dots, \theta_d$ steht $\frac{\partial}{\partial \theta_i}$ für die Ableitung nach der Variablen θ_i ; dabei werden alle anderen Variablen in Bezug auf diese Ableitung als Konstanten behandelt
- Z.B. gilt für $f(\theta_0, \dots, \theta_d) \equiv f(\theta) \coloneqq \sum_{i=0}^{d} \theta_i x_i = \theta_0 x_0 + \cdots + \theta_d x_d$:
  - $\frac{\partial f(\theta)}{\partial \theta_i} = x_i$ - partielle Ableitung nach θ_i , $\quad i = (1, \dots, d)$
- Def. Gradient $\nabla_\theta f(\theta) \equiv \begin{pmatrix} \frac{\partial f}{\partial \theta_0} \\ \vdots \\ \frac{\partial f}{\partial \theta_d} \end{pmatrix}$ (für reellwertige Funktion f)
- $\nabla_\theta f(\theta)$ zeigt Richtung der stärksten Steigung der Funktion $f(\theta)$
- lokale Minima (oder Maxima) liegen bei $\nabla_\theta f(\theta) = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$

Supervised Learning

Supervised Learning Pictogram

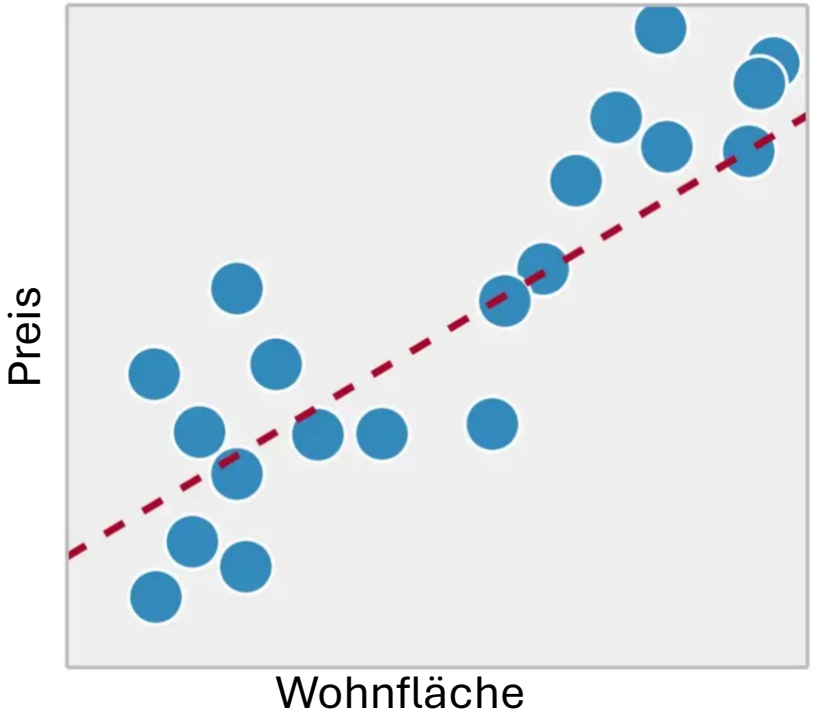


neue Daten x, z.B. Wohnfläche eines Hauses, das nicht Teil der Trainingsdaten ist.

Vorhersage y für neue Daten x, z.B. Preis eines Hauses, das nicht Teil der Trainingsdaten ist.

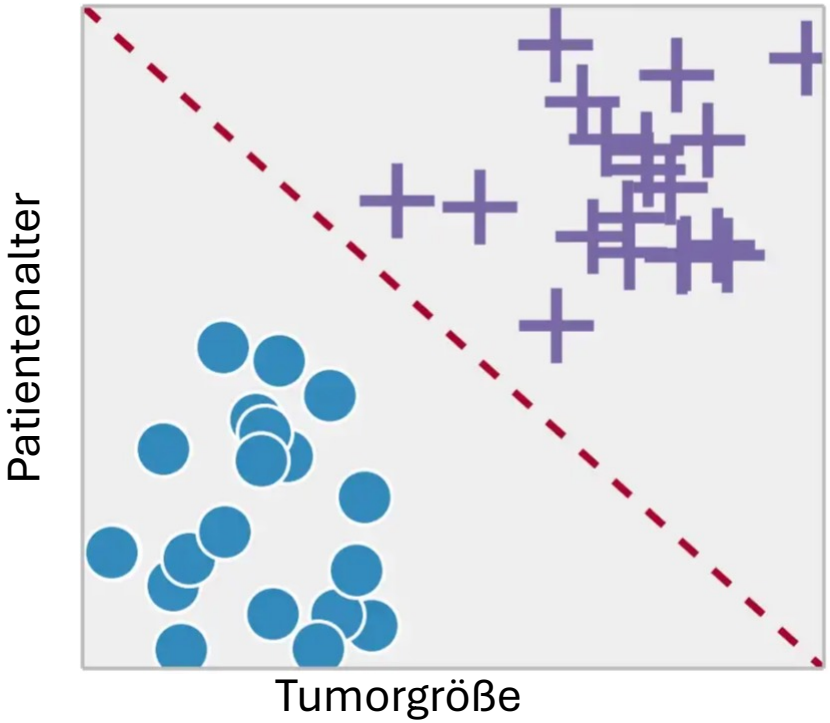
Supervised Learning

Beispiel 1: Hauspreise



Regression: Input → reelle Zahl

Beispiel 2: Krebs



Klassifikation: Input → ganze Zahl (“Klasse” bzw. “Label”)

Notation

Input $x^{(i)}$ („features“)

Output $y^{(i)}$ („target“)

Living area (feet^2)	#bedrooms	Price (1000$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
⋮	⋮	⋮

n: #Trainingsdaten

d: #Features (hier: $d = 2$)

$(x^{(i)}, y^{(i)})$: Einzelnes Trainings-Beispiel mit Index (i)

$\{(x^{(i)}, y^{(i)}); i = 1, \dots, n\}$: Menge der Trainingsdaten

X, Y: Raum der Input (respektive Output) Werte; im Beispiel oben $X = \mathbb{R}^2, Y = \mathbb{R}$

Supervised Learning Problemstellung:

Gegeben Trainingsdatensatz, finde eine Funktion $h\colon X \to Y$, sodass $h(x)$ eine „gute“ Näherung für das zu x gehörige y ist.

h wird auch **Hypothese** („hypothesis“) genannt.

Kostenfunktion

- wie wählen („lernen“) wir θ bei gegebenen Trainingsdaten?
- versuchen $h_\theta(x)$ so nah wie möglich an y aus den Trainingsdaten zu bringen
- Formal: Definition einer **Kostenfunktion**, die für jede Wahl von θ misst, wie weit $h_\theta(x)$ von y entfernt ist (x, y sind Vektoren, d.h. sie enthalten die gesamten Trainingsdaten):

Kostenfunktion

$$\boxed{J(\theta) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2}$$

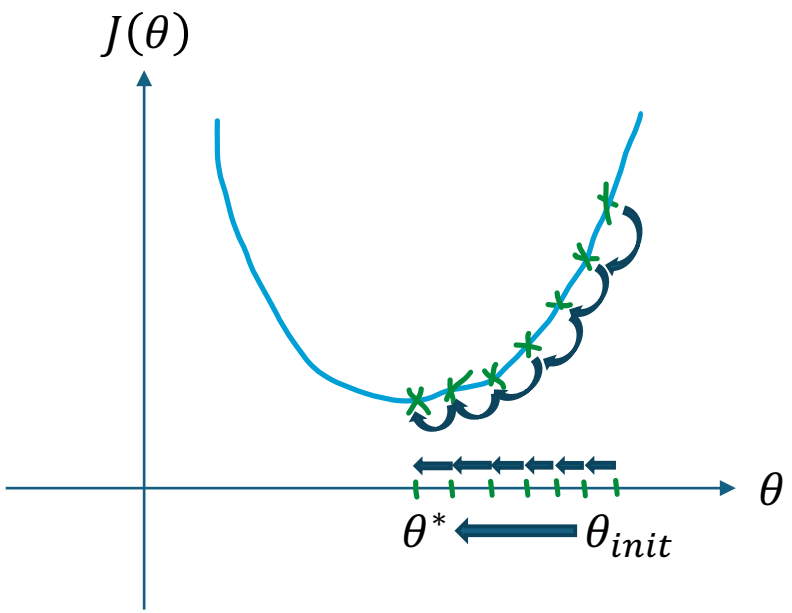
n: Anzahl der Trainingsdaten

- Aufgabe: Wähle θ so, dass $J(\theta)$ minimiert wird

- “Methode der kleinsten Quadrate” oder “least-squares cost function”

Gradient Descent: Numerische Optimierung

- bei der numerischen Minimierung von $J(\theta)$ startet man mit einem beliebigen Wert für θ, θ_{init}, und passt diesen iterativ an, bis man „nah genug“ an der Lösung ist
- Dies ist der „Gradient Descent“ Algorithmus
- Beispiel mit $d = 1$ (nur 1 Feature):



Gradient Descent: Numerische Optimierung

- Der Algorithmus Gradient Descent berechnet den Gradienten

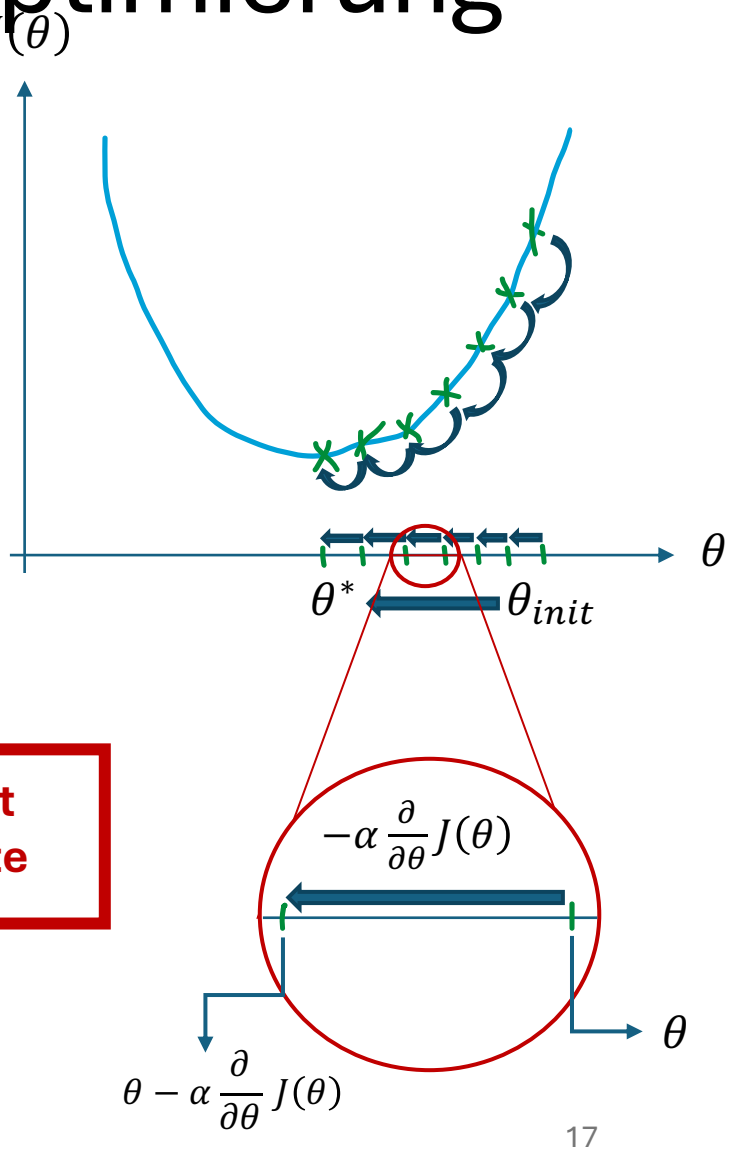
$$\nabla_\theta J(\theta) \equiv \begin{pmatrix} \frac{\partial J}{\partial \theta_0} \\ \vdots \\ \frac{\partial J}{\partial \theta_d} \end{pmatrix}$$

- $\nabla_\theta J(\theta)$ zeigt die Richtung des stärksten Anstiegs von $J(\theta)$ an; in diese Richtung wird θ verändert:

$$\boxed{\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad (\mathrm{j} = 0, \dots, \mathrm{d})}$$

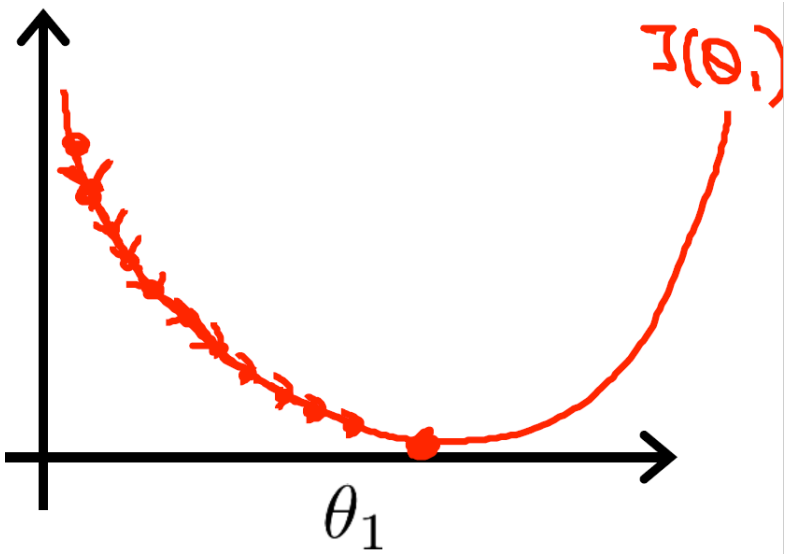
Gradient Descent Parameter Update

- Die Lernrate („learning rate“) α ist ein Parameter des Gradient Descent Algorithmus‘

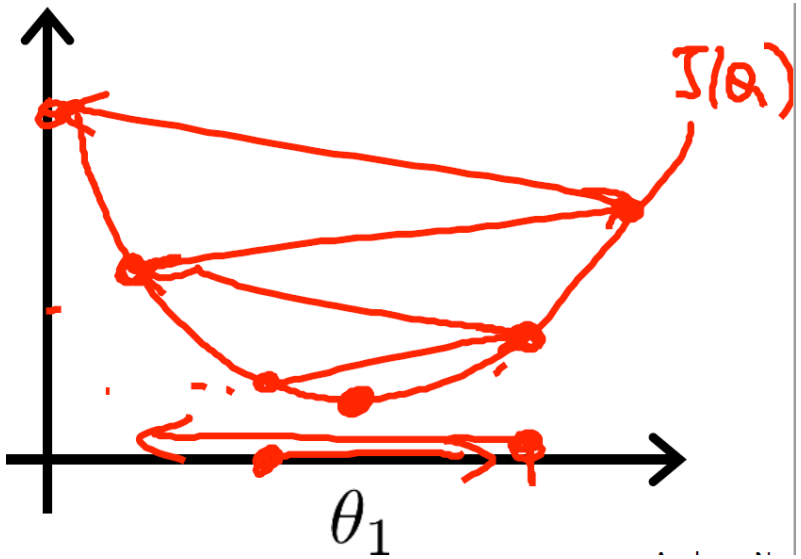


Gradient Descent – Wahl der Lernrate α

Ist α zu klein, kann Gradient Descent sehr lange dauern bis es konvergiert



Ist α zu groß, kann es passieren, dass Gradient Descent nicht konvergiert oder sogar divergiert

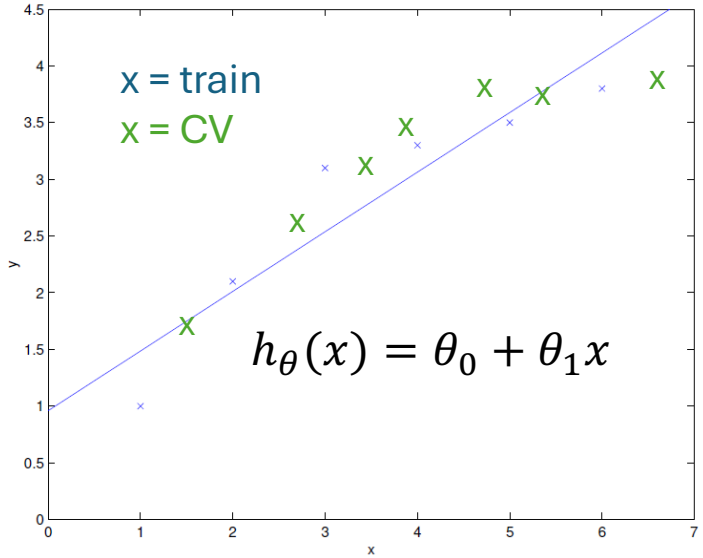


Es gibt viele Weiterentwicklungen von Gradient Descent, die sich u.a. mit der dynamischen Wahl von α beschäftigen.

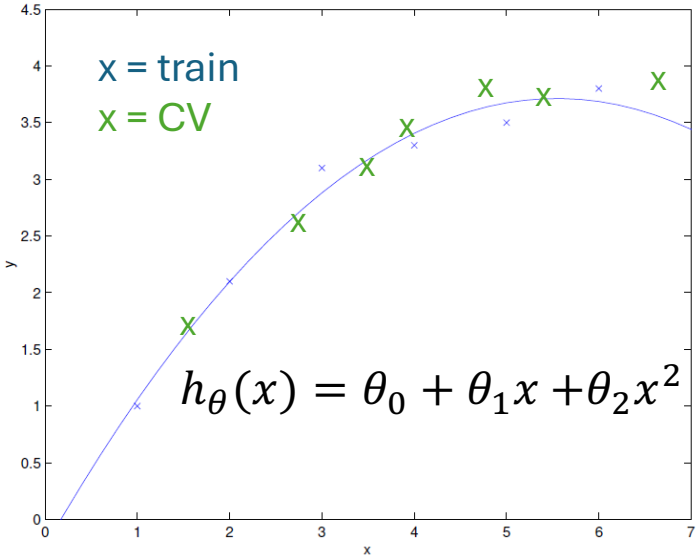
Modell-Evaluation

Bias & Variance

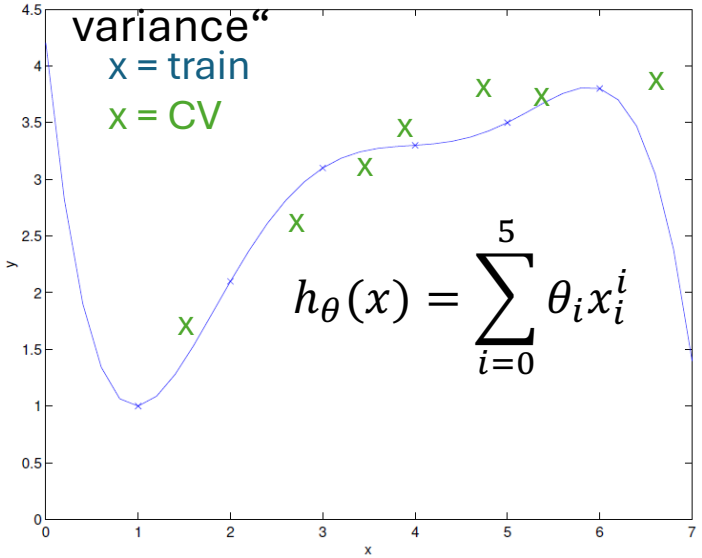
„underfitting“ / „high bias“



„OK“



„overfitting“ / „high variance“

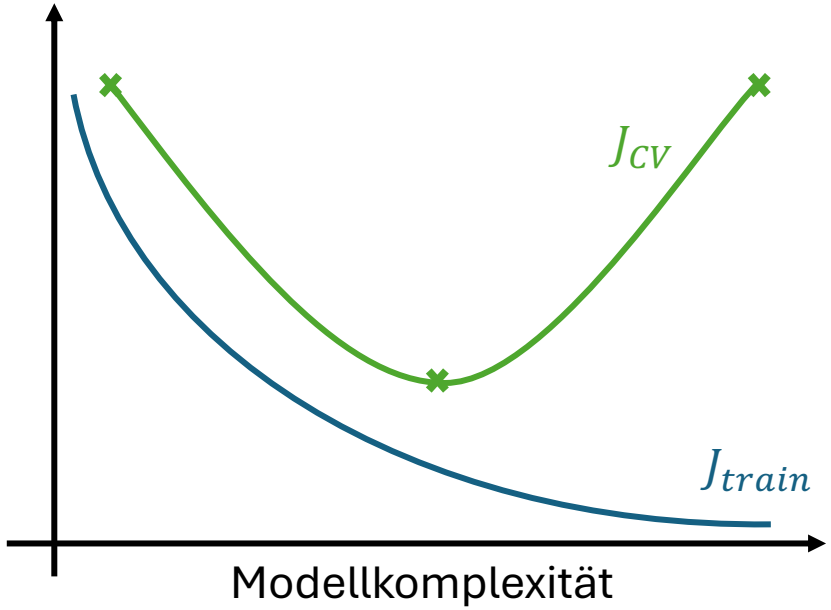


Wenn wir viele Features haben, können wir die Vorhersage nicht als Funktion aller Features plotten; stattdessen nutzen wir J_{train} und J_{CV} zur Diagnose:

J_{train} ist groß	J_{train} ist klein	J_{train} ist klein
J_{CV} ist groß	J_{CV} ist klein	J_{CV} ist groß

Bias & Variance verstehen

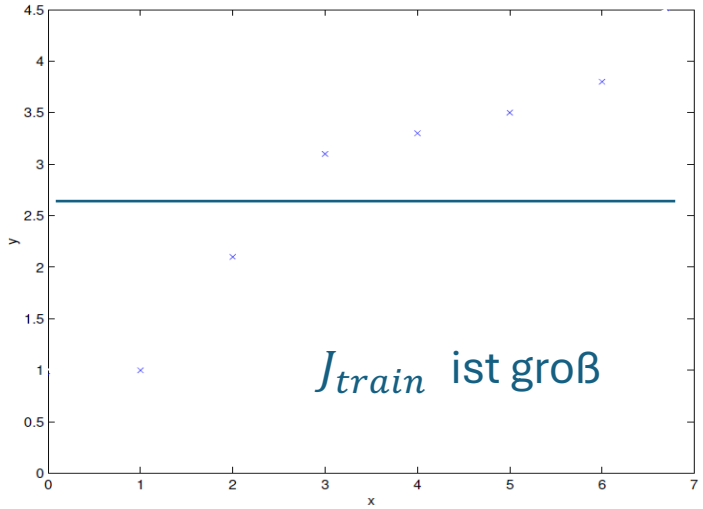
Wir können J_{train} und J_{CV} als Funktion eines Modell-Hyperparameters bzw. für verschiedene untersuchte Modelle untersuchen:



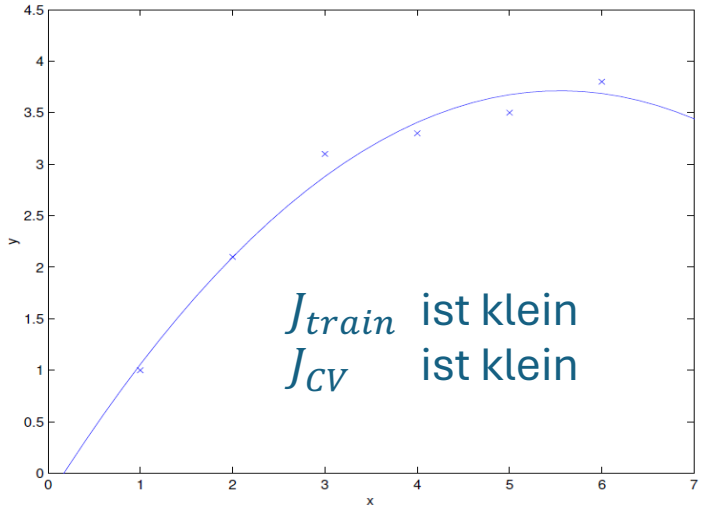
Auf diese Weise finden wir das Modell mit dem optimalen Tradeoff zwischen hohem Bias und hoher Varianz.

Bias & Variance und Regularisierung

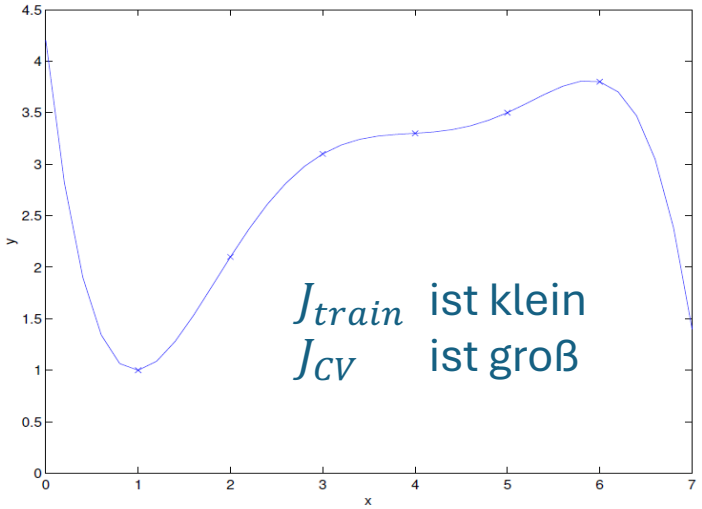
Modell: $h_\theta(x) = \sum_{i=0}^{5} \theta_i x_i^i, \quad J(\theta) = \frac{1}{2n_{train}} \left(\sum_{i=1}^{n_{train}} \left(h_\theta \left(x_{train}^{(i)} \right) - y_{train}^{(i)} \right)^2 + \lambda \sum_{j=1}^{d} \theta_j^2 \right)$



Großes λ
Hoher Bias



Mittleres λ

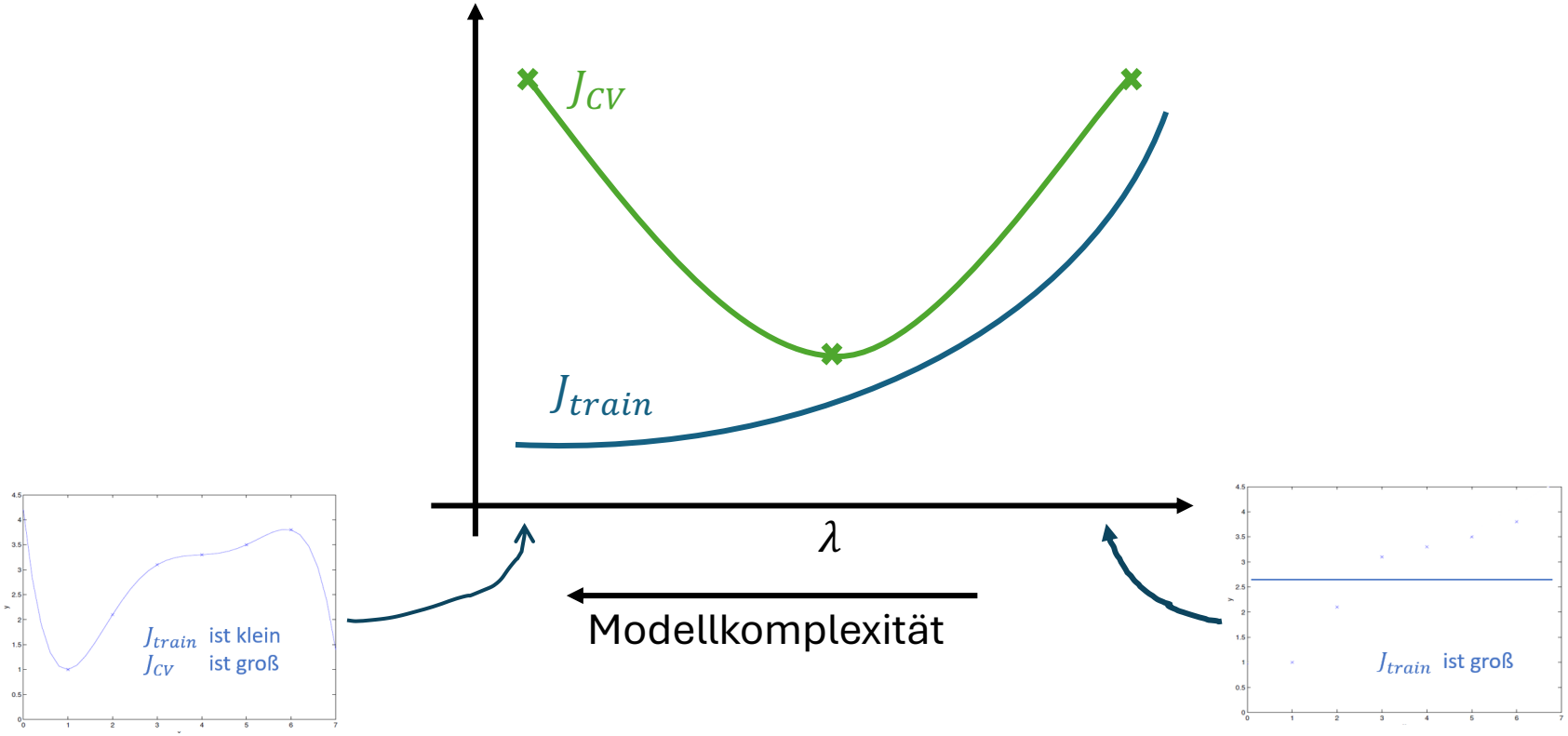


Kleines λ
Hohe Varianz

Weitere Regularisierungsmethoden für neuronale Netzwerke sind unter anderem **Dropout, Early Stopping, Batch Normalization und Layer Normalization**.

Bias und Varianz als Funktion von λ

$$h_\theta(x) = \sum_{i=0}^{5} \theta_i x_i^i, \quad J(\theta) = \frac{1}{2n_{train}} \left(\sum_{i=1}^{n_{train}} \left(h_\theta \left(x_{train}^{(i)} \right) - y_{train}^{(i)} \right)^2 + \lambda \sum_{j=1}^{d} \theta_j^2 \right)$$

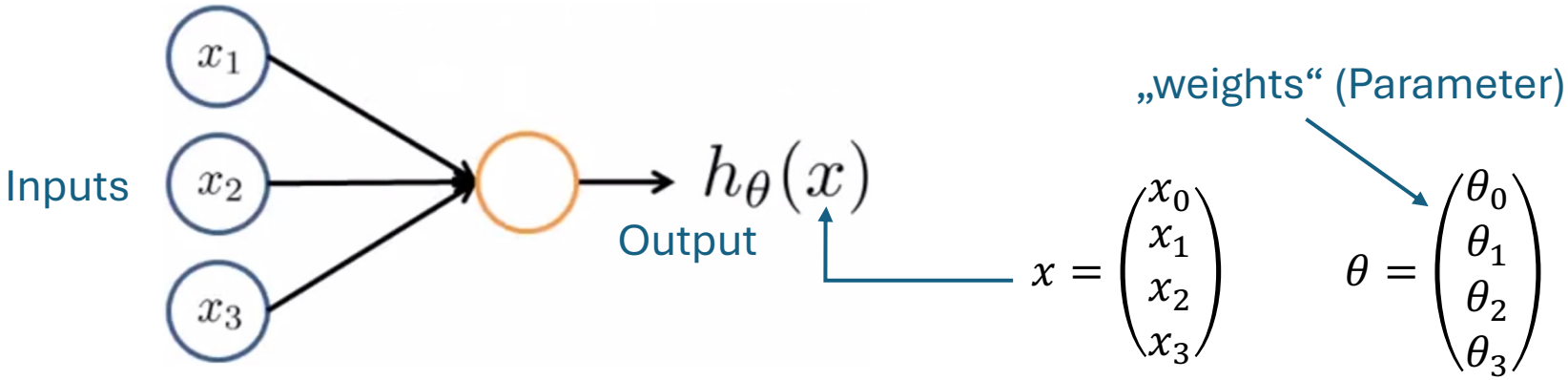


Fully Connected Netzwerke

Auch: Multilayer Perceptron (MLP), Dense Networks

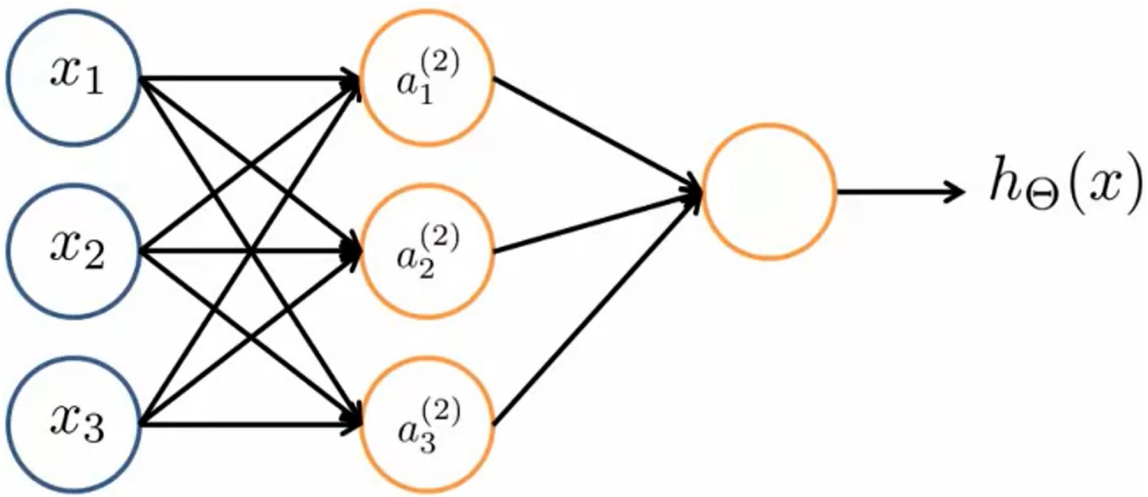
Künstliches Neuron

- Modellieren ein Neuron als einfache Recheneinheit:



- $h_\theta(x) = g(\theta^T x)$
- g ist die sogenannte „Aktivierungsfunktion“ (oder“Aktivierung“), z.B. die logistische Funktion (Sigmoid): $g(\theta^T x) = \frac{1}{1+e^{-\theta^T x}}$, ReLU, tanh
- Input $x_0 = 1$ implizit immer dabei, auch wenn nicht eingezeichnet

Multilayer Perceptron (MLP)



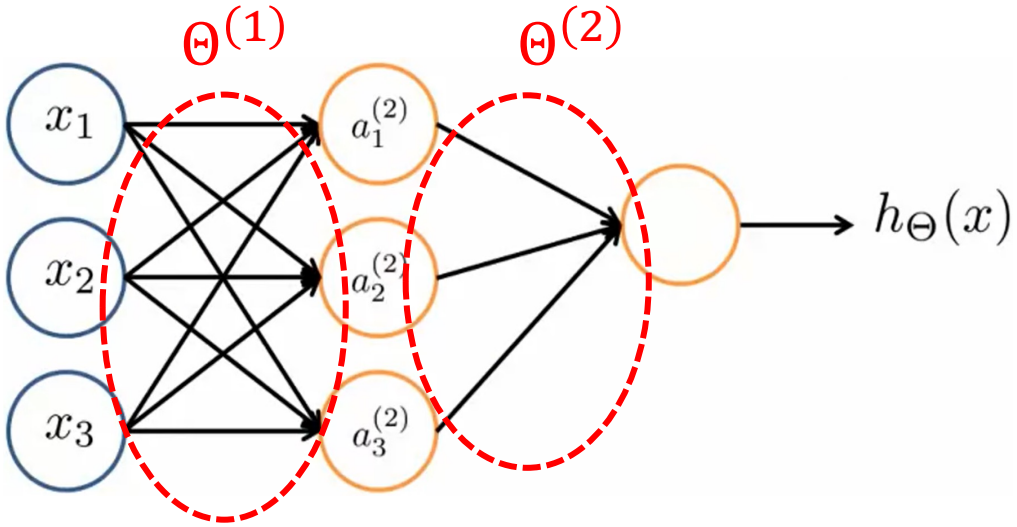
Layer 1
„Input Layer“

Layer 2
„Hidden Layer“

Layer 3
„Output Layer“

- Auch: Fully Connected oder Dense Network
- Schichtweiser Aufbau
- alle Inputs mit allen “Neuronen” im ersten Hidden Layer verbunden
- Zwischen zwei Schichten alle Neuronen von einer Schicht mit allen Neuronen der folgenden Schicht verbunden
- jedes “Neuron” a_i hat seinen eigenen Parametervektor “θ^{a_i}“
- Man nennt die Neuronen $a_i^{(j)}$ auch **Aktivierungen**
- Auch hidden layers haben implizit stets ein $a_0^{(j)} = 1$ in jedem hidden layer
- Beliebig viele Hidden Layer möglich

Notation



Layer 1
„Input Layer“

Layer 2
„Hidden Layer“

Layer 3
„Output Layer“

$z_i^{(l)}$: Input der Aktivierung i in Layer l

$a_i^{(l)}$: Output der Aktivierung i in Layer l

$\Theta_i^{(l)}$: Parametervektor, der den Input von Layer l zum Output des Neurons $a_i^{(l+1)}$ von Layer $l+1$ abbildet: $a_i^{(l+1)} = g(\Theta_i^{(l)} x)$

→ wenn man das Neuron $a_i^{(l+1)}$ als eigene Einheit von logistischer Regression betrachtet, ist $\Theta_i^{(l)}$ sein Parametervektor (exakt wie bei logistischer Regression)

Organisiert man alle Parametervektoren $\Theta_i^{(l)}$ als Zeilen untereinander, erhält man die Matrix $\Theta^{(l)}$

Für das Netzwerk oben gilt damit:

$$a_1^{(2)} = g(\Theta_1^{(1)} x) = g\left(\overbrace{\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3}^{z_1^{(2)}}\right)$$

$$a_2^{(2)} = g(\Theta_2^{(1)} x) = g\left(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3\right)$$

$$a_3^{(2)} = g(\Theta_3^{(1)} x) = g\left(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3\right)$$

$$h_\Theta(x) = a_1^{(3)} = g\left(\Theta_1^{(2)} a^{(2)}\right) = g\left(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)}\right)$$

Bei q_l Einheiten in Layer l und q_{l+1} Einheiten in Layer $l+1$ ist $\Theta^{(l)}$ eine $q_{l+1} \times (q_l + 1)$ Matrix.

(Die “+1” ist wegen des Bias-Terms $x_0 = a_0^{(k)} = 1$).

Training des Netzwerks (3)

$$\frac{\partial J_k}{\partial \Theta_{ij}^{(l)}} = \delta_{ki}^{(l+1)} a_j^{(l)}$$

$$\frac{\partial z_s^{(l+1)}}{\partial z_i^{(l)}} = ? \; ; \quad z_s^{(l+1)} = \sum_m \Theta_{sm}^{(l)} a_m^{(l)} = \sum_m \Theta_{sm}^{(l)} g\left(z_m^{(l)}\right) \quad \rightarrow$$

$$\frac{\partial z_s^{(l+1)}}{\partial z_i^{(l)}} = \frac{\partial\left(\sum_m \Theta_{sm}^{(l)} g\left(z_m^{(l)}\right)\right)}{\partial z_i^{(l)}} = \Theta_{si}^{(l)} \frac{\partial g\left(z_i^{(l)}\right)}{\partial z_i^{(l)}} \equiv \Theta_{si}^{(l)} g'\left(z_i^{(l)}\right) \quad \rightarrow$$

$$\delta_{ki}^{(l)} = \sum_s \delta_{ks}^{(l+1)} \frac{\partial z_s^{(l+1)}}{\partial z_i^{(l)}} = \sum_s \delta_{ks}^{(l+1)} \Theta_{si}^{(l)} g'\left(z_i^{(l)}\right) \; , \quad \text{bzw:}$$

$$\delta_k^{(l)} = \left(\left(\Theta^{(l)}\right)^T \delta_k^{(l+1)}\right) * g'\left(z^{(l)}\right)$$

Backpropagation

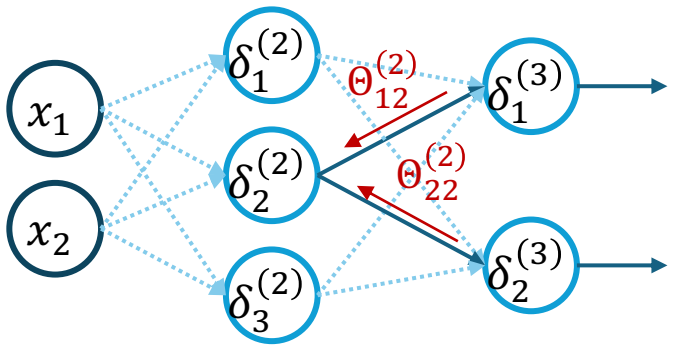
komponentenweise Multiplikation,
also: $(a * b)_i = a_i b_i$

Backpropagation anschaulich

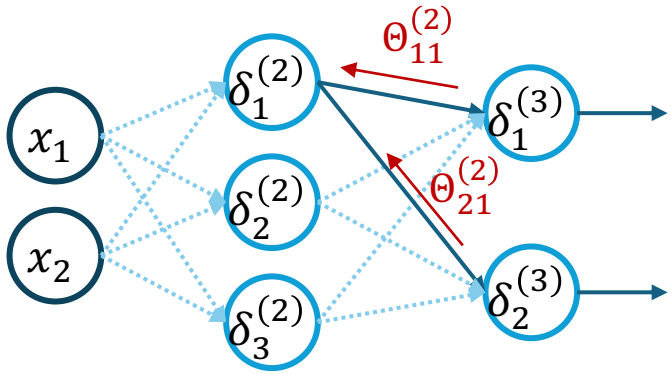
$$\frac{\partial J_k}{\partial \Theta_{ij}^{(l)}} = \delta_{ki}^{(l+1)} a_j^{(l)}$$

- $\delta_{ki}^{(l)} = \sum_s \delta_{ks}^{(l+1)} \Theta_{si}^{(l)} g'\left(z_i^{(l)}\right), \ \delta_{ki}^{(l)} := \frac{\partial J_k}{\partial z_i^{(l)}}$
- $\delta_{k1}^{(3)} = \frac{\partial J_k}{\partial z_1^{(3)}}, \ \ \delta_{k2}^{(3)} = \frac{\partial J_k}{\partial z_2^{(3)}}$

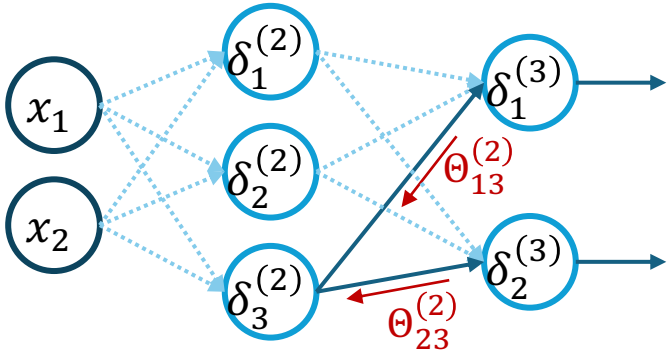
$$\delta_{k2}^{(2)} = \left(\delta_{k1}^{(3)} \Theta_{12}^{(2)} + \delta_{k2}^{(3)} \Theta_{22}^{(2)}\right) g'(z_2^{(2)})$$



$$\delta_{k1}^{(2)} = \left(\delta_{k1}^{(3)} \Theta_{11}^{(2)} + \delta_{k2}^{(3)} \Theta_{21}^{(2)}\right) g'(z_1^{(2)})$$



$$\delta_{k3}^{(2)} = \left(\delta_{k1}^{(3)} \Theta_{13}^{(2)} + \delta_{k2}^{(3)} \Theta_{23}^{(2)}\right) g'(z_3^{(2)})$$



Softmax am Output für Mehrklassenklassifizierung

- Bei mehreren Klassen nutzt man oft Softmax am Output um eine Wahrscheinlichkeits-Interpretation der Vorhersagen zu ermöglichen:

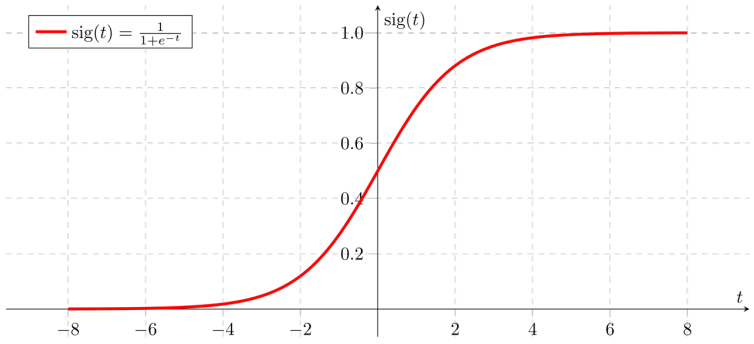
$$z = \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix} \quad \rightarrow \quad \mathrm{softmax}(z_i) := \frac{e^{z_i}}{\sum_{k=1}^{n} e^{z_k}} \rightarrow \sum_{i=1}^{n} \mathrm{softmax}(z_i) = 1$$

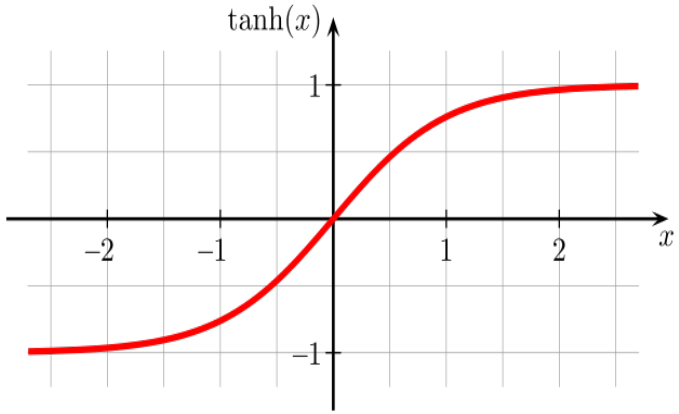
Beispiel:

$$z = \begin{pmatrix} 1 \\ 2 \\ 4 \end{pmatrix} \quad \rightarrow \quad \mathrm{softmax}(z) = \begin{pmatrix} 0.042 \\ 0.114 \\ 0.844 \end{pmatrix}$$

Aktivierungsfunktionen

- früher beliebt: $\mathrm{sigmoid}(x) = \frac{1}{1+e^{-x}} \in [0; 1]$ oder
- $\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}} \in [-1; +1]$ („Tangens hyperbolicus“)





- Problem bei beiden: „Saturieren“ und Output verändert sich kaum, wenn Input $x \gg 0$ oder $x \ll 0$
- → entsprechend wird der Gradient sehr klein und Netzwerk kann nicht effektiv trainiert werden; aber: Wertebereiche für RNNs nützlich
- $g'(z) \in (0; 1]$ für tanh oder sigmoid